

Introdução a Algoritmos

Aula 2 – Variáveis, Tipos e Operações

Raoni F. S. Teixeira · 1s/2026

Objetivos desta aula

Ao final desta aula, você deverá ser capaz de:

1. Declarar variáveis com o tipo adequado ao problema (int, float, double, char) e justificar a escolha do tipo;
2. Prever o resultado de expressões aritméticas com operadores de precedências diferentes, incluindo divisão inteira e o operador %;
3. Identificar e corrigir o erro de divisão inteira silenciosa usando casting explícito;
4. Ler e escrever valores de diferentes tipos com scanf e printf, usando os especificadores de formato corretos.

1 A peça “dados”: guardar informação

Na Aula 1 apresentamos as peças fundamentais da programação. Entre elas, “dados”: armazenar e organizar informação na memória. É a peça mais rica do curso — variável, vetor, matriz, struct e ponteiro são todas formas de “dados”, em níveis crescentes de complexidade. Nesta aula estudamos a forma mais simples: a **variável**.

Imagine que você está resolvendo uma conta longa no papel. Ao longo dos cálculos, você anota resultados intermediários para usá-los mais tarde — escreve “subtotal = 47” numa folha e consulta esse valor quando precisa. Em um programa, as variáveis cumprem exatamente esse papel: são espaços na memória do computador onde armazenamos valores que precisaremos usar ou modificar durante a execução.

Definição — Variável

Uma variável é um **nome** associado a uma **posição de memória** que armazena um valor de um determinado **tipo**. O valor pode ser lido e modificado ao longo da execução do programa.

Cada variável tem três atributos essenciais:

Atributo	Significado	Exemplo
Nome	Como chamamos a variável no código	idade
Tipo	Que categoria de valor ela armazena	int
Valor	O conteúdo atual armazenado	21

2 Por que tipos existem?

À primeira vista, pode parecer desnecessário distinguir entre números inteiros, números com casas decimais e caracteres — afinal, tudo na memória do computador é representado por bits. Por que não usar um único tipo universal?

A resposta tem duas dimensões: **eficiência** e **correção**.

Em termos de eficiência, representar um número inteiro pequeno como 5 com 64 bits (8 bytes) quando 8 bits (1 byte) seriam suficientes é um desperdício de memória. Num programa com milhões de valores, esse desperdício se acumula.

Em termos de correção, tipos protegem o programador de erros silenciosos. Se você tentar armazenar o número 3.14 numa variável inteira, o computador simplesmente descartará a parte decimal e guardará apenas 3 — sem aviso, sem erro. Conhecer os tipos evita esse tipo de surpresa.

2.1 Tipos inteiros

Usados para armazenar números sem parte fracionária.

Tipo	Tamanho típico	Faixa de valores
int	32 bits	-2 147 483 648 a +2 147 483 647
unsigned int	32 bits	0 a 4 294 967 295
short int	16 bits	-32 768 a +32 767
long int	64 bits	$\pm 9,2 \times 10^{18}$ (aproximadamente)
char	8 bits	-128 a +127 (ou 0 a 255 sem sinal)

Atenção

O tipo char armazena um único caractere, mas internamente é um número inteiro de 8 bits. O caractere 'A', por exemplo, é armazenado como o número 65 (código ASCII). Escrever `char c = 65;` e `char c = 'A';` são equivalentes em C.

Exemplo — Overflow — o que acontece quando o valor excede a faixa

```
#include <stdio.h>
int main() {
    short int x = 32767; /* valor máximo de short int */
    x = x + 1;
    printf("%d\n", x);    /* imprime -32768, não 32768! */
    return 0;
}
```

O valor “dá a volta” e passa para o extremo negativo. Esse fenômeno chama-se **overflow** e é uma das fontes mais comuns de bugs difíceis de encontrar. A lição: escolha o tipo correto para a magnitude dos valores que seu programa manipula.

2.2 Tipos de ponto flutuante

Usados para números reais — aqueles com parte fracionária.

Tipo	Tamanho	Precisão aproximada
float	32 bits	6–7 dígitos significativos
double	64 bits	15–16 dígitos significativos

Ponto flutuante não é matemática exata. O número 0.1 não tem representação binária exata, assim como 1/3 não tem representação decimal exata. Por isso, expressões como `0.1 + 0.2 == 0.3` podem ser **falsas** em C. Para comparar valores em ponto flutuante, verifica-se se a diferença entre eles é suficientemente pequena, não se são exatamente iguais.

2.3 Regras para nomes de variáveis

Nomes de variáveis devem ser **descritivos** — `velocidade_inicial` é muito melhor que `vi` ou `x3`. Além disso, C impõe algumas restrições sintáticas:

- Devem começar por uma letra ou sublinhado (`_`);
- Não podem começar por número;
- Podem conter letras, números e sublinhados;
- São sensíveis a maiúsculas/minúsculas (`Soma` e `soma` são variáveis diferentes);
- Palavras reservadas da linguagem não podem ser usadas (`int`, `if`, `while`, etc.).

3 Declaração e atribuição

Em C, toda variável deve ser **declarada** antes de ser usada. A declaração informa ao compilador o nome e o tipo da variável, reservando o espaço de memória necessário.

```
int idade;  
float preco;  
char inicial;
```

A declaração apenas reserva espaço — ela não define um valor inicial. Uma variável declarada mas não inicializada contém um valor **indeterminado** (qualquer “lixo” que estivesse naquele endereço de memória antes). Usar esse valor é um erro de lógica silencioso.

O operador de **atribuição** = armazena um valor numa variável:

```
idade = 21;  
preco = 9.99;  
inicial = 'R';
```

É possível declarar e inicializar na mesma linha — e isso é considerado boa prática:

```
int idade = 21;  
float preco = 9.99;  
char inicial = 'R';
```

Atenção

O operador `=` em C não é igualdade matemática — é uma **instrução de cópia**. A expressão `x = x + 1` é perfeitamente válida: ela lê o valor atual de `x`, soma 1, e armazena o resultado de volta em `x`. Em matemática, essa equação não tem solução; em programação, é apenas “incremente `x`”.

4 Estrutura básica de um programa em C

Com variáveis em mãos, podemos escrever programas que guardam e manipulam informação. A estrutura padrão é:

```
#include <stdio.h>

int main() {
    /* 1. Declaração de variáveis */
    int a, b, resultado;

    /* 2. Entrada de dados */
    scanf("%d %d", &a, &b);

    /* 3. Processamento */
    resultado = a + b;

    /* 4. Saída */
    printf("Soma: %d\n", resultado);

    return 0;
}
```

Essa divisão em quatro partes — declaração, entrada, processamento, saída — não é obrigatória em C, mas é uma organização que torna o código mais fácil de ler e depurar. Adote-a como hábito desde o início.

5 Saída de dados: printf

A função `printf` exibe texto e valores na tela. Ela recebe uma **string de formato** contendo texto fixo e **especificadores** que indicam onde e como os valores devem ser inseridos.

```
int n = 42;
float x = 3.14;
printf("n vale %d e x vale %f\n", n, x);
/* saída: n vale 42 e x vale 3.140000 */
```

Os especificadores mais usados são:

Especificador	Tipo correspondente
%d	inteiro (int, short, char)
%u	inteiro sem sinal (unsigned int)
%ld	inteiro longo (long int)

%f	ponto flutuante (float)
%lf	ponto flutuante duplo (double)
%c	caractere (char)
%s	cadeia de caracteres (string)

O `\n` dentro da string representa uma quebra de linha. Outros **caracteres de escape** úteis: `\t` (tabulação), `\\` (barra invertida literal), `\"` (aspas literais).

Para controlar a formatação numérica, podemos especificar a largura e a precisão:

```
printf("%.2f\n", 3.14159); /* saída: 3.14 – 2 casas decimais */
printf("%8d\n", 42);      /* saída:      42 – campo de 8 caracteres */
```

6 Entrada de dados: scanf

A função `scanf` lê valores digitados pelo usuário e os armazena em variáveis. Sua sintaxe é semelhante à do `printf`, mas há uma diferença crucial: é necessário passar o **endereço** de cada variável, usando o operador `&`.

```
int n;
printf("Digite um número: ");
scanf("%d", &n);
printf("Você digitou: %d\n", n);
```

Por que o `&`? Porque `scanf` precisa saber **onde** na memória armazenar o valor lido. Sem o `&`, passaríamos o valor atual da variável (que pode ser lixo), não sua posição de memória. Esse é um dos erros mais comuns entre iniciantes — e o compilador nem sempre avisa.

Para ler múltiplas variáveis de uma vez:

```
int a, b, c;
scanf("%d %d %d", &a, &b, &c);
```

7 Operações aritméticas

Variáveis por si só não fazem muito — precisamos **operá-las** para produzir resultados. C oferece os operadores aritméticos fundamentais:

Operador	Operação	Observação
+	Adição	
-	Subtração	
*	Multiplicação	
/	Divisão	Comportamento depende dos tipos (ver abaixo)
%	Resto da divisão	Válido apenas para operandos inteiros

7.1 A divisão e seus dois comportamentos

O operador `/` tem um comportamento que surpreende muitos iniciantes: quando ambos os operandos são inteiros, ele realiza **divisão inteira**, descartando a parte fracionária.

```
int a = 10, b = 3;
printf("%d\n", a / b);           /* imprime 3, não 3.333... */
printf("%f\n", a / (float)b);   /* imprime 3.333333 */
```

A expressão `(float)b` é uma **conversão explícita de tipo** (casting): ela instrui o compilador a tratar `b` como `float` naquele cálculo. Quando pelo menos um operando é de ponto flutuante, a divisão passa a ser real.

Exemplo — Armadilha clássica da divisão inteira

```
float media;
int soma = 7, n = 2;
media = soma / n;           /* media = 3.0, não 3.5! */
media = (float)soma / n;    /* media = 3.5 – correto */
```

A primeira linha faz divisão inteira **antes** de atribuir à variável `float`. O casting precisa ser aplicado **antes** da divisão.

7.2 Precedência de operadores

Quando uma expressão contém múltiplos operadores, a ordem de avaliação segue regras de precedência — as mesmas da álgebra:

1. `*`, `/` e `%` são avaliados da esquerda para a direita;
2. `+` e `-` são avaliados depois, também da esquerda para a direita;
3. Parênteses alteram a ordem e têm precedência máxima.

```
int x = 8 + 10 * 6;           /* x = 68   (não 108) */
int y = (8 + 10) * 6;         /* y = 108          */
int z = 5 + 10 % 3;           /* z = 6   (10%3 = 1, depois 5+1) */
```

Use parênteses generosamente. Sempre que a ordem de avaliação não for completamente óbvia, adicione parênteses. Eles não custam nada e evitam bugs sutis causados por precedência inesperada.

8 Atalhos e simplificações

C oferece uma série de atalhos para operações comuns:

8.1 Operadores de atribuição composta

```
a += b;   /* equivale a: a = a + b; */
a -= b;   /* equivale a: a = a - b; */
a *= b;   /* equivale a: a = a * b; */
a /= b;   /* equivale a: a = a / b; */
a %= b;   /* equivale a: a = a % b; */
```

8.2 Incremento e decremento

As operações de somar ou subtrair 1 de uma variável são tão frequentes que C tem operadores próprios para isso:

```
a++; /* a = a + 1 */
a--; /* a = a - 1 */
```

Esses operadores existem em versão **pré-fixada** (++a) e **pós-fixada** (a++). A diferença importa quando o operador é usado dentro de uma expressão:

```
int a = 10;
printf("%d\n", ++a); /* imprime 11 – incrementa ANTES de usar */
printf("%d\n", a++); /* imprime 11 – usa ANTES de incrementar (a passa a ser 12) */
```

Atenção

Evite usar ++ e -- dentro de expressões complexas. A expressão `x = a * ++a` é tecnicamente **indefinida** em C — o compilador pode avaliá-la de formas diferentes. Use esses operadores apenas como instruções isoladas: `i++;` em vez de `x = f(i++);`.

9 Conversão de tipos

Quando operandos de tipos diferentes aparecem numa mesma expressão, C realiza conversões automaticamente — às vezes de forma surpreendente.

9.1 Conversão implícita (promoção)

C converte automaticamente o tipo “menor” para o tipo “maior” quando isso não causa perda de informação:

```
int i = 5;
float f = i; /* i é promovido para float: f = 5.0 */
```

A hierarquia geral é: `char` → `short` → `int` → `long` → `float` → `double`.

9.2 Conversão explícita (casting)

Quando queremos forçar uma conversão — mesmo que haja perda de informação — usamos casting:

```
float x = 3.99;
int n = (int)x; /* n = 3 – parte decimal descartada silenciosamente */
```

O casting não arredonda: ele **trunca**. O valor 3.99 vira 3, não 4.

10 Exemplo integrador

O programa abaixo calcula a média ponderada de duas notas, ilustrando o uso conjunto de variáveis, entrada, operações e saída:

```
#include <stdio.h>
```

```
/*
```

```
* Descrição: Lê duas notas e seus pesos e calcula a média ponderada.
* Entrada:  nota1, peso1, nota2, peso2 (valores reais)
* Saída:    média ponderada
* Pré-cond.: pesos positivos, notas no intervalo [0, 10]
*/
int main() {
    float nota1, nota2, peso1, peso2, media;

    printf("Nota 1 e peso 1: ");
    scanf("%f %f", &nota1, &peso1);

    printf("Nota 2 e peso 2: ");
    scanf("%f %f", &nota2, &peso2);

    media = (nota1 * peso1 + nota2 * peso2) / (peso1 + peso2);

    printf("Média ponderada: %.2f\n", media);
    return 0;
}
```

Observe a documentação no início — ela segue o padrão que adotaremos em todo o curso: descrição, entradas, saídas e pré-condições.

11 Exercícios

1. Declare variáveis adequadas e escreva um programa que leia o raio de um círculo e imprima sua área e seu perímetro com duas casas decimais. (Use $\pi \approx 3.14159$.)
2. O que imprime o programa abaixo? Explique por quê, sem executá-lo:

```
int a = 7, b = 2;
printf("%d %f\n", a / b, (float)a / b);
```

3. Analise a expressão: $1 + 2 * 30 / (++n \% 4 * -5)$ para $n = 3$. (a) Indique a ordem em que as operações são executadas.
(b) Reescreva o programa usando uma operação por linha.
(c) Existe um valor de n para o qual esse programa falha? Qual e por quê?